

Chapitre 10 Variables dynamiques

Contrairement aux variables statiques, dont la réservation en mémoire est décidée par le programmeur au moment de l'écriture du code, les variables dynamiques sont réservées en mémoire et libérées au fur et à mesure de l'exécution des programmes, selon les besoins.

Elles utilisent l'opérateur `new` de réservation mémoire et l'opérateur `delete` de libération mémoire, étroitement associés aux variables pointeurs (abordés au Chapitre 7 page 41).

Opérateur `new`

L'opérateur `new` crée une variable dynamique dont le type et la valeur (spécifiée entre parenthèses) sont donnés par le programmeur, et retourne un pointeur sur cette variable (ou la valeur `NULL` si la réservation de la mémoire a échoué).

Syntaxe

```
new type;           // réserve un emplacement mémoire et renvoie son adresse
```

ou

```
new type (valeur); // initialise en plus cet emplacement mémoire
```

ou

```
new type [n];       // réserve un emplacement mémoire de taille n (int) × type
```

Remarques

- L'opérateur `new` réalise au moins deux et au plus 3 opérations :
 - 1) `new` réserve (si la mémoire libre est suffisante) un emplacement mémoire de la taille de `type`, ou de `n (int)` fois la taille de `type` si `n` est indiqué entre crochets `[n]` à la suite de `type`
 - 2) Éventuellement, `new` initialise l'emplacement mémoire par la valeur indiquée entre parenthèses (optionnel)
 - 3) `new` retourne l'adresse de l'emplacement mémoire alloué (ou `NULL` si une erreur est survenue lors de la tentative d'allocation)
- Il est courant de stocker l'adresse de l'emplacement mémoire dynamique réservé par `new` dans une variable statique de type pointeur :

```
type* pPointeur = new type;
```

ou

```
type* pPointeur = new type (valeur);
```

ou

```
int nN = 16;
type* pPointeur = new type [nN];
```

- L'adresse de la variable dynamique retournée par `new` doit être conservée pour utiliser cet emplacement dynamique.

- Comme pour toutes les variables, c'est au programmeur de veiller à l'initialisation des variables dynamiques. Dans ce cas il faut penser à initialiser à la fois le contenu du pointeur sur la variable dynamique et la variable dynamique elle-même.
- Toujours vérifier si la valeur retournée par `new` n'est pas `NULL` avant d'utiliser la variable dynamique :

```
double* pMois = new double (12);
if(pMois != NULL)
{
    // utilisation de la variable dynamique
}
```

Au minimum, utiliser `ASSERT (pPointeur==NULL)` juste après `new`. `ASSERT(condition)` retourne 0 et stoppe proprement l'exécution du programme si la condition est fausse.

```
ASSERT(pMois == NULL);
```

- L'entier `n` passé entre crochets `[n]` n'est pas nécessairement constant.
- Si à ce stade il peut paraître peu utile de créer des variables dynamiques dont les adresses sont stockées dans des pointeurs statiques, on verra plus loin que les variables dynamiques sont très utiles pour manipuler des tableaux (dynamiques) dont la taille dépend d'un nombre de données choisi pendant l'exécution des programmes.

Exemple

```
double* pPI = new double; // pointeur statique et double dynamique
*pPI = 3.141592654;       // initialisation de la variable dynamique

double* pE = new double (2.71828); // plus rapide, plus rationnel plus sûr

int nN = 8;
char* pChaine = new char [nN];
```

Opérateur `delete`

A l'inverse de `new`, l'opérateur `delete` libère la mémoire d'une variable dynamique.

Syntaxe

```
delete pointeur; // si allocation d'une variable dynamique simple
```

ou

```
delete [] pointeur; // si allocation d'une variable dynamique créée avec [nN]
```

Remarques

- L'opérande de l'opérateur `delete` doit être un pointeur initialisé par `new`.
- Pour éviter de générer une erreur d'exécution, il faut toujours tester si la valeur du pointeur passé à l'opérateur `delete` est différente de la valeur `NULL` :

```
if (pMem != NULL)
    delete pMem ;
```

- Lorsque la variable dynamique créée est multiple, il est indispensable d'utiliser l'opérateur `delete [ ] pTab`.

Exemple (suite de l'exemple précédent)

```
delete pI; // libération de la variable dynamique dont l'adresse est dans pI
delete pE; // libération de la variable dynamique dont l'adresse est dans pE
delete[] pChaine; // utilisation de delete[]; car allocation par new char[nN]
```

Différence entre variables statiques pointeurs et variables dynamiques

Une variable dynamique est une variable exclusivement créée par l'opérateur `new`. Ceci signifie que toute autre variable créée sans l'opérateur `new` est une variable statique.

Exemple

```
double*    pE = NULL; // variable statique pointeur pE
int*       pI = NULL; // variable statique pointeur pI
pE = ← new double (3.1416); // variable dynamique dont l'adresse est dans pE
pI = ← new int (8); // variable dynamique int dont l'adresse est dans pI
```

- `pE, pI` sont des variables statiques pointeurs, qui contiennent des adresses, codées sur 8 octets (retournées par `new`)
- `*pE` est une variable dynamique de type `double`, qui contient un réel (3.1416) codé sur 8 octets
- `*pI` est une variable dynamique de type `int`, qui contient un entier (8), codé sur 8 octets (processeur 64 bits).

